

Public transport ticket pricing system

This is a copy of a chat between Claude and laura. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. [Report](#)

I would like you to generate me a function in python that can be used in an app used for public transport. The function should allow for the tickets to be bought in different versions, for a single ride, for set period and in bundles. The tickets should be able to contain different prices depending on the needs, that is, prices should differ if the person is a student, senior or a regular person. This feature should be usable through mobile application or other interface. This feature should be able to determine the price of a ticket depending on its type. It should also allow the user to purchase multiple tickets at once, which the function should be able to handle as well, even if the tickets have different discounts and different ranges, the system should be able to accumulate the price without any errors. If for example customer chooses ticket without specifying the quantity, the system should throw an error. The function should be able to handle multiple requests at the same time even with heavy workflow. On top of it customers personal information should be kept secret as well as payment details. Thank you

Show less

Apr 28 

Viewed a file, created a file, read a file >

Here's a breakdown of every design decision made in the module:

Architecture

The system is built around a single class, `TransitTicketingSystem`, which exposes three public methods: `get_ticket_price`, `calculate_cart_total`, and `purchase`. This makes it easy to wire up to any interface — a REST endpoint, a mobile SDK, a web form, etc.

Ticket types & passenger tiers

Both are Python `Enum`s (`TicketType` and `PassengerTier`), so the caller can never pass a typo. Prices and discount multipliers live in simple dicts at the top of the file, making them trivially easy to update from a database or config service in production.

Quantity validation

`TicketLineItem` is a frozen dataclass. Its `__post_init__` method immediately raises a `TicketQuantityError` if `quantity` is `None`, zero, or negative — before any pricing logic ever runs. This satisfies the "error on missing quantity" requirement cleanly.

Mixed-cart accumulation

`calculate_cart_total` iterates over any number of line items with different types, tiers, durations, and bundle sizes, accumulates `Decimal` subtotals to avoid floating-point drift, and returns both the per-line subtotals and a rounded grand total.

Thread safety

A `threading.Lock` guards the shared audit log, so concurrent requests from a mobile app or API gateway can't corrupt state. The pricing and calculation paths are stateless and need no locking.

PII & payment security

- `CustomerInfo.from_raw()` accepts a full name and raw card number but **never stores either**. The name is reduced to initials (`"Anna Kowalska"` → `"A.K."`) and the card is masked (`"***** * 4242"`). The raw PAN is SHA-256 hashed for the payment gateway stub and then discarded.
- `PurchaseReceipt` only ever contains the masked card and the customer UUID — nothing a bad actor could exploit.
- The payment gateway call is isolated in `_authorise_payment`, making it straightforward to swap in Stripe, Adyen, or any other provider without touching the business logic.

